3 Less Typing

Since the beginning of time, Man has had an uneasy relationship with his keyboard. Sure, keyboards make it possible to express our precise wishes to the computer, but in our fat-fingered excitement to get stuff done, we often suffer from typos and digital fatigue.

In this adventure, we will travel down the carpal tunnel to the land of less typing. We covered some of this in TLCL, but here we will look a little deeper.

Aliases and Shell Functions

The first thing we can do to reduce the number of characters we type is to make full use of *aliases* and *shell functions*. Aliases were created for this very purpose and they are often a very effective solution. Shell functions perform in many ways like aliases but allow a full range of shell script-like capabilities such as programmatic logic, and option and argument processing.

Most Linux distributions provide some set of default alias definitions and it's easy to add more. To see the aliases we already have, we enter the `alias` command without arguments:

```
me@linuxbox: ~ $ alias
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias ls='ls --color=auto
```

On this example system, we see `alias` is used to activate color output for some commonly used commands. It is also common to create aliases for various forms of the `ls` command:

```
alias ll='ls -l'
alias la='ls -A'
alias l='ls -CF'
alias l.='ls -d .*'
alias lm='ls -l | less'
```

Aliases are good for lots of things, for example, here's one that's useful for Debian-style systems:

```
alias update='sudo apt-get update && sudo apt-get upgrade'
```

Aliases are easy to create. It's usually just a matter of appending them to our `.bashrc` file. Before creating a new alias, it's a good idea to first test the proposed name of the alias with the `type` command to check if the name is already being used by another program or alias.

While being easy, aliases are somewhat limited. In particular, aliases can't handle complex logic or accept positional parameters. For that we need to use shell functions.

As we recall from TLCL, shell functions are miniature shell scripts that we can add to our `.bashrc` file to perform anything that we may otherwise do with a shell script. Here is an example function that displays a quick snapshot of a system's health:

```
status() {
  { echo -e "\nuptime:"
    uptime
    echo -e "\ndisk space:"
    df -h 2> /dev/null
    echo -e "\ninodes:"
    df -i 2> /dev/null
    echo -e "\nblock devices:"
    blkid
    echo -e "\nmemory:"
    free -m
    if [[ -r /var/log/syslog ]]; then
      echo -e "\nsyslog:"
      tail /var/log/syslog
    fi
    if [[ -r /var/log/messages ]]; then
      echo -e "\nmessages:"
      tail /var/log/messages
    fi
  } | less
}
```

Unlike aliases, shell functions can accept positional parameters:

```
params() {
    local argc=0
    while [[ -n $1 ]]; do
        argc=$((++argc))
        echo "Argument $argc = $1"
        shift
    done
}
```

Command Line Editing

Aliases and shell functions are all well and good, provided we know in advance the operations we wish to perform, but what about the rest of the time? Most command line operations we perform are on-the-fly, so other techniques are needed.

As we saw in Chapter 8 of TLCL, the bash shell includes a library called *readline* to handle keyboard input during interactive shell sessions. This includes text typed at the shell prompt and keyboard input using the `read` builtin when the `-e` option is specified. The readline library supports a large number of commands that can be used to edit what we type at the command line. Since readline is from the GNU project, many of the commands are taken from the emacs text editor.

Control Commands

Before we get to the actual editing commands, let's look at some commands that are used to control the editing process.

Command	Description
`Enter`	Pressing the enter key causes the current command line to be accepted. Note that the cursor location within the line does not matter (i.e., it doesn't have to be at the end). If the line is not empty, it is added to the command history.
`Esc`	Meta-prefix. If the `Alt` key is unavailable, the `Esc` key can be used in its place. For example, if a command calls for `Alt-r` but another program intercepts that command, press and release the `Esc` key followed by the `r` key.
`Ctrl-g`	Abort the current editing command.
`Ctrl-_`	Incrementally undo changes to the line.
`Alt-r`	Revert all changes to the line (i.e., complete undo).
`Ctrl-l`	Clear the screen.
`Alt-num`	Where `num` is a number. Some commands accept a numeric argument. For those commands that accept it, type this first followed by the command.

Editing control commands

Moving Around

Here are some commands to move the cursor around the current command line. In the readline documentation, the current cursor location is referred to as the *point*.

Command	Description
`Right`	Move forward one character.
`Left`	Move backward one character.
`Alt-f`	Move forward one word.
`Alt-b`	Move backward one word.
`Ctrl-a`	Move to the beginning of the line.
`Ctrl-e`	Move to the end of the line.

Cursor movement commands

Using Command History

In order to save typing, we frequently reuse previously typed commands stored in the command history. We can move up and down the history list and the history list can be searched.

Command	Description
`Up`	Move to previous history list entry.
`Down`	Move to next history list entry.

Command	Description
`Alt-<`	Move to the beginning of the history list.
`Alt->`	Move to the end of the history list.
`Ctrl-r`	Perform an incremental history search starting at the current position and moving up the history list. After a command is typed, a prompt appears and with each succeeding character typed, the position within the list moves to the next matching line. This is probably the most useful of the history search commands.
`Ctrl-s`	Like `Ctrl-r` except the search is performed moving down the history list.
`Alt-p`	Perform a non-incremental search moving up the history list.
`Alt-n`	Perform a non-incremental search moving down the history list.
`Alt-Ctrl-y`	Insert the first argument from the previous history entry. This command can take a numeric argument. When a numeric argument is given, the nth argument from the previous history entry is inserted.
`Alt-.`	Insert the last argument from the previous history entry. When a numeric argument is given, behavior is the same as `Alt-Ctrl-y` above.

History commands

Changing Text

Command	Description
`Ctrl-d`	Delete the character at the point.
`Ctrl-t`	Transpose characters. Exchange the character at the point with the character preceding it.
`Alt-t`	Transpose words. Exchange the word at the point with the word preceding it.
`Alt-u`	Change the current word to uppercase.
`Alt-l`	Change the current word to lowercase.
`Alt-c`	Capitalize the current word.

Editing commands

Cutting and Pasting

As with `vim`, cutting and pasting in readline are referred to as “killing” and “yanking.” The clipboard is called the *kill-ring* and is implemented as a *circular buffer*. This means that it contains multiple entries (i.e., each kill adds a new entry). The latest entry is referred to as the “top” entry. It is possible to “rotate” the kill-ring to bring the previous entry to the top and delete the latest entry. However, this feature is rarely used.

Mostly, the kill commands are used to simply delete text rather than save it for later yanking.

Command	Description
`Alt-d`	Kill from the point to the end of the current word. If the point is located in whitespace, kill to the end of the next word.
`Alt-Backspace`	Kill the word before the point.
`Ctrl-k`	Kill from the point to end of line.
`Ctrl-u`	Kill from the point to the beginning of the line.
`Ctrl-y`	Yank the “top” entry from the kill-ring.
`Alt-y`	Rotate the kill-ring and yank the new “top” entry.

Copy and delete commands

Editing in Action

In Chapter 4 of TLCL, we considered the danger of using a wildcard with the `rm` command. It was suggested that we first test the wildcard with the `ls` command to see the result of the expansion. We then recall the command from the history and edit the line to replace the “ls” with “rm”. So, how do we perform this simple edit?

First, the beginner’s way: we recall the command with the up arrow, use the left arrow repeatedly to move the cursor to the space between the “ls” and the wildcard, backspace twice, then type “rm” and `Enter`.

That’s a lot of keystrokes.

Next, the tough-guy’s way: we recall the command with the up arrow, type `Ctrl-a` to jump to the beginning of the line, type `Alt-d` to kill the current word (the “ls”), type “rm” and `Enter`.

That’s better.

Finally, the super-tough-guy’s way: type “rm” then `Alt-.` to recall the last argument (the wildcard) from the previous command, then `Enter`.

Wow.

Completion

Another trick that readline can perform is called *completion*. This is where readline will attempt to automatically complete something we type.

For example, let’s imagine that our current working directory contains a single file named `foo.txt` and we want to view it with `less`. So we begin to type the command `less foo.txt` but instead of typing it all out, we just type `less f` and then press the `Tab` key. Pressing `Tab` tells readline to attempt completion on the file name and remainder of the command is completed automatically.

This will work as long as the “clue” given to readline is not ambiguous. If we had two files in our imaginary directory named “foo.txt” and “foo1.txt”, a successful completion would not take place since “less f” could refer to either file. What happens instead is readline makes the next best guess by completing as far as “less foo” since both possible answers contain those characters. To make a full completion, we need to type either `less foo.` for `foo.txt` or `less foo1` for `foo1.txt`.

If we have typed an ambiguous clue, we can view a list of all possible completions to get guidance as what to type next. In the case of our imaginary directory, pressing `Tab` a second time will display all of the file names beginning with “foo” so that we can see what more needs to be typed to remove the ambiguity.

Besides file name completion, readline can complete command names, environment variable names, user home directory names, and network host names:

Completion	**Description**
Command names	Completion on the first word of a line will complete the name of an available command. For example, typing “lsu” followed by `Tab` will complete as `lsusb`.
Variables	If completion is attempted on a word beginning with “$”, environment variable names will be used. For example, typing “echo $TE” will complete as `echo $TERM`.
User names	To complete the name of a user’s home directory, precede the user’s name with a “~” and press ‘Tab’. For example: `ls ~ro` followed by `Tab` will complete to `ls ~root/`. It is also possible to force completion of a user name without the leading `~` by typing `Alt-~`. For example “who ro” followed by `Alt-~` will complete to `who root`.
Host names	Completion on a word starting with “@” causes host name completion, however this feature rarely works on modern systems since they tend to use DHCP rather than listing host names in the `/etc/hosts` file.
File names	In all other cases, completion is attempted on file and path names.

Completion types

Programmable Completion

Bash includes some builtin commands that permit the completion facility to be programmed on a command-by-command basis. This means it’s possible to set up a custom completion scheme for individual commands; however, doing this is beyond the scope of this adventure. We will instead talk about an optional package that uses these builtins to greatly extend the native completion facility. Called *bash-completion*, this package is installed automatically for some distributions (for example, Ubuntu) and is

generally available for others. To check for the package, examine the `/etc/bash-completion.d` directory. If it exists, the package is installed.

The bash-completion package adds support for many command line programs, allowing us to perform completion on both command options and arguments. The `ls` command is a good example. If we type "ls –" then the `Tab` key a couple of times, we will see a list of possible options to the command:

```
me@linuxbox: ~ $ ls --
--all                                        --ignore=
--almost-all                                 --ignore-backups
--author                                     --indicator-style=
--block-size=                                --inode
--classify                                   --literal
--color                                      --no-group
--color=                                     --numeric-uid-gid
--context                                    --quote-name
--dereference                                --quoting-style=
--dereference-command-line                   --recursive
--dereference-command-line-symlink-to-dir    --reverse
--directory                                  --show-control-chars
--dired                                      --si
--escape                                     --size
--file-type                                  --sort
--format=                                    --sort=
--group-directories-first                    --tabsize=
--help                                       --time=
--hide=                                      --time-style=
--hide-control-chars                         --version
--human-readable                             --width=
```

An option can be completed by typing a partial option followed by `Tab`. For example, typing "ls –ver" then `Tab` will complete to "ls –version".

The bash-completion system is interesting in its own right as it is implemented by a series of shell scripts that make use of the `complete` and `compgen` bash builtins. The main body of the work is done by the `/etc/bash_completion` (or `/usr/share/bash-completion/bash_completion` in newer versions) script along with additional scripts for individual programs in either the `/etc/bash-completion.d` directory or the `/usr/share/bash-completion/completions` directory. These scripts are good examples of advanced scripting technique and are worthy of study.

Summing Up

This adventure is a lot to take in and it might not seem all that useful at first, but as we continue to gain experience and practice with the command line, learning these labor-saving tricks will save us a lot of time and effort.

Further Reading

- "The beginning of time" actually has meaning in Unix-like operating systems such as Linux. It's January 1, 1970. See: https://en.wikipedia.org/wiki/Unix_time for details.
- Aliases and shell functions are discussed in Chapters 5 and 26, respectively, of *The Linux Command Line*: https://linuxcommand.org/tlcl.php.
- The READLINE section of the bash man page describes the many keyboard shortcuts available on the command line.
- The HISTORY section of the bash man page covers the command line history features of `bash`.
- The official home page of the bash-completion project: https://github.com/scop/bash-completion
- For those readers interested in learning how to write their own bash completion scripts, see this tutorial at the Linux Documentation Project: https://tldp.org/LDP/abs/html/tabexpansion.html.